

Report conversation

Is it more gas efficient to pack types smaller than uint256 together in a Solidity contract storage?

E.g. is contract B more gas efficient than B?

```
solidity
pragma solidity >=0.8.19;

contract A {
    uint128 public x;
    uint256 public y;
    uint128 public z;
}

contract B {
    uint128 public x;
    uint128 public y;
    uint256 public z;
}
```

In Solidity, storage layout optimization can indeed save gas costs, but the specific example you provided doesn't necessarily result in significant gas savings just by rearranging the variables. This is because each storage slot in Ethereum costs the same amount of gas to write to or read from, regardless of the data type. Each storage slot is 32 bytes (or 256 bits), and Ethereum's EVM (Ethereum Virtual Machine) operates on 256-bit words.

In your example:

- Contract A has uint128 , uint256 , and uint128 variables.
- Contract B has uint128 , uint128 , and uint256 variables.

Both contracts will use the same number of storage slots:

- Contract A: 1 slot for x , 1 slot for y , and 1 slot for z = 3 slots
- Contract B: 1 slot for x , 1 slot for y , and 1 slot for z = 3 slots

However, if you were to pack smaller variables together into a single storage slot, you could save gas. Solidity allows you to do this manually by using structs. Here's an example:

solidity

 Copy code

```
pragma solidity >=0.8.19;

contract C {
    struct PackedData {
        uint128 x;
        uint128 y;
    }

    PackedData public data;
    uint256 public z;
}
```

In this example, contract C uses only 2 storage slots: one for the `PackedData` struct (`x` and `y` are packed into a single 256-bit word) and one for `z` . This would be more gas-efficient when it comes to storage costs.

So, while your original contracts A and B don't differ in terms of storage gas efficiency, you can use techniques like struct packing to optimize storage costs.